

Filosofía y Mentalidad Ágil

-

La Agilidad como Ideología

- Es una filosofía de trabajo y toma de decisiones orientada a maximizar la entrega continua de valor en entornos de alta incertidumbre, priorizando la adaptación empírica sobre la planificación, y la autonomía sobre el control burocrático de los procesos.
-

Manifiesto Ágil

- Documento creado para dar un nombre formal a un enfoque de desarrollo y establecer una declaración conjunta de
- valores y principios rectores.

Valores

- Los **Individuos e Interacciones sobre Procesos y Herramientas** indica que un equipo cohesionado de buenas personas supera a otro equipo disfuncional con buenas herramientas y procesos.
- El **Software Funcionando sobre Documentación Extensiva** indica la prioridad de entregar valor real que funcione en cada iteración en vez de documentación perfecta.
- La **Colaboración con el Cliente sobre la Negociación Contractual** fomenta el diálogo y comunicación para guiar el producto en vez de un contrato estático.
- **Responder al Cambio sobre Seguir el Plan** indica que los requisitos cambiantes no se evitan, se aceptan como una ventaja competitiva, ajustando el rumbo.
-

Principios

- **Satisfacer al Cliente** implica que la mayor prioridad es satisfacer al cliente mediante la entrega temprana

y continua de software con valor.

- **Aceptar los Cambios** implica que los requisitos cambiantes se reciben de buen agrado, incluso en etapas avanzadas, para beneficio competitivo del cliente.
 - Las **Entregas Frecuentes** implican liberar software funcional periódicamente
 - El **Trabajo Conjunto Diario** implica que los responsables del negocio y los desarrolladores deben trabajar juntos todos los días durante todo el proyecto.
 - Tener **Individuos Motivados** implica construir los proyectos alrededor de personas motivadas, dándoles el entorno, el apoyo y la confianza necesarios para hacer el trabajo.
 - La **Conversación Cara a Cara** es el método más eficiente y efectivo para comunicar información dentro del equipo.
 - El **Software Funcionando como Medida de Progreso** sostiene que el software que realmente funciona es la métrica principal para medir el avance del proyecto.
 - El **Desarrollo Sostenible** promueve un ritmo de trabajo constante e indefinido para promotores, desarrolladores y usuarios (evitando el agotamiento por horas extra).
 - La **Excelencia Técnica** y a buen diseño mejora y potencia la agilidad.
 - La **Simplicidad** es esencial para maximizar la cantidad de trabajo no realizado.
 - Los **Equipos Autoorganizados** generan las mejores arquitecturas, requisitos y diseños
 - La **Reflexión y Ajuste Continuo** implica que a intervalos regulares, el equipo reflexiona sobre cómo ser más eficaz y sintoniza su comportamiento en consecuencia.
- .

Enfoques Adaptativos vs Predictivos

- Los **Enfoques Predictivos** están dirigidos por un plan

rígido, partiendo de la premisa de que todo es predecible, busca definir los requisitos al inicio del proyecto y priorizando el cumplimiento del plan por sobre la flexibilidad ante los cambios.

- Los **Enfoques Adaptativos** están dirigidos por el valor entregado al cliente, aceptando que en la mayoría de los proyectos es imposible conocer todos los detalles al inicio, por lo que se usa una planificación continua y entregas frecuentes para ajustar el rumbo.
-

Los Procesos Empíricos

- Modelo que se basa en la experiencia, la observación de hechos reales y la experimentación directa en lugar de análisis abstractos.
- Funciona mediante un ciclo constante de planificar, hacer y adaptar, construyendo las piezas de software reales para obtener retroalimentación rápida de los usuarios y aprender
-

Principales Frameworks Ágiles



Scrum

- Es un marco de trabajo enfocado principalmente en las prácticas de organización y gestión del equipo.
- El progreso del proyecto se divide en ciclos cortos de tiempo fijo llamados Sprints.
- En cada Sprint un equipo autoorganizado trabaja en conjunto guiado por un Product Owner (quien prioriza el valor del negocio) y un Scrum Master (facilita el proceso)



XP (Extreme Programming)

- Marco de trabajo enfocado en las prácticas técnicas y de ingeniería de software.
- Su objetivo es asegurar la excelencia técnica y la calidad interna del producto mediante disciplinas rigurosas como el TDD, CI/CD y refactorización constante



Gestión de Requerimientos Ágiles y Lean

•

El Costo Tradicional de BRUF (Big Requirements Up Front)

- Es el costo y desperdicio generado por intentar definir, analizar y documentar de manera exhaustiva el % de los requerimientos al inicio de un proyecto.
- El **Desperdicio** producido por BRUF es del 64% de la funcionalidad construida, la cual el 45% no se usa nunca y el 19% rara vez se usa.

•

Priorización Dinámica de la Pila - Product Backlog

- Es el ordenamiento continuo de todos los requerimientos o ítems de trabajo en una lista única y dinámica, donde los elementos de mayor importancia se posicionan en la parte superior.
- Sirve para maximizar el valor de negocio entregado al cliente en cada iteración y permitir que los cambios de requerimientos actúen como una ventaja competitiva en vez de ser un problema
- de ser un problema

Funcionamiento

1. Los Requerimientos de **Alta Prioridad** se ubican arriba y se analizan con mucho detalle
2. Los Requerimientos de **Baja Prioridad** se ubican abajo y se mantienen de forma muy abstracta
3. En cualquier momento del proyecto el Product Owner puede re-priorizar,, agregar o eliminar elementos de la pila según cambien las necesidades del mercado.

•

Análisis Just In Time (JIT)

- Es la práctica de diferir el análisis detallado de los requerimientos lo más tarde posible, idealmente justo antes de que el equipo comience a desarrollarlos.
- Evita analizar o documentar por adelantado características que podrían sufrir cambios drásticos o que incluso nunca lleguen a programarse.

- Permite capturar las necesidades en formatos sencillos y el análisis se realiza al final con la información más actualizada del proyecto.

Niveles de Requerimientos

- Un **Theme (Tema)** es un grupo o colección de User Stories relacionadas que comparten un objetivo común de negocio
- Una **Epic (Épica)** es una User Story muy grande que describe una necesidad general del cliente, y debe ser descompuesta en historias más pequeñas que entren en una iteración.
- Una **User Story (Historia de Usuario)** es una descripción concisa y de alto nivel de una funcionalidad deseada, contada desde la perspectiva del usuario o cliente.

Principios Lean Aplicados a los Requerimientos



Eliminate Waste

- Identificar y erradicar cualquier actividad que consuma recursos pero no aporte valor al cliente
- En requerimientos consiste en no generar documentación masiva ni especificaciones teóricas innecesarias



Defer Commitment

- Retrasar la toma de decisiones críticas o el detalle minucioso de requerimientos hasta el último momento posible.
- Basar las decisiones en aprendizaje real y no en especulaciones inicial



See the Whole

- Comprender el sistema en su totalidad.
 - Asegurar que cada requerimiento individual se diseñe entendiendo cómo impacta en el flujo de valor general del producto
-
-

Fundamentos de User Stories

- Una **User Story** es una descripción breve y de alto nivel de una funcionalidad que aporta valor al usuario o comprador del producto.
- No es una especificación detallada ni un requerimiento rígido, sino más bien una expresión de intención que sirve como un recordatorio físico para una conversación futura.
- Funciona como invitación para fomentar la comunicación directa entre el equipo y el cliente, ayudando a planificar y estimar el trabajo de forma ágil sin caer en documentación excesiva.

Las 3 Cs - Card, Conversation, Confirmation

- Una **Card** es el soporte físico de la historia, que contiene una descripción muy breve y sirve para planificar y recordar que existe una necesidad, representando el requerimiento.
- Una **Conversation** es el diálogo continuo y cara a cara entre desarrolladores, clientes y usuarios para acordar y profundizar en los detalles específicos de la funcionalidad en el momento exacto en que se va a construir.
- Una **Confirmation** son las pruebas de aceptación que plasman de forma concreta los criterios que demuestran que la historia ha sido terminada correctamente y funciona según las expectativas del negocio.

Estructura Clásica de Redacción

- Como `<rol>`, quiero `<actividad>` para `<valor>`
- El **Rol** representa al tipo de usuario individual que interactúa con el sistema o recibe el beneficio directo de la acción.
- La **Actividad** describe la acción o comportamiento concreto que el sistema realizará o que el usuario podrá ejecutar.
- El **Valor** comunica la necesidad de negocio o la razón de ser de la funcionalidad, dejando en claro qué beneficio real obtiene el usuario o cliente con ella.

Porciones Verticales (Vertical Slices)

- Es una técnica de división de requerimientos que evita separar el trabajo de forma horizontal.
 - Consiste en diseñar e implementar historia de usuario que atraviesen todas las capas arquitectónicas del sistema necesarias para completar esa funcionalidad específica.
 - Su propósito es garantizar que cada entrega del equipo sea un incremento de software funcional de extremo a extremo, permitiendo al cliente ver resultados reales que funcionan, probarlos y liberar valor de forma continua y segura.
-

Modelado de Usuarios y Roles

- Es el proceso de identificar, definir y agrupar a los distintos usuarios finales de un sistema en perfiles específicos, caracterizándolos por atributos comunes como su frecuencia de uso, su experiencia técnica, su nivel de conocimiento del negocio y sus objetivos al usar el sistema.
- Sirve para evitar el error de diseñar bajo la premisa de un único usuario promedio genérico.
- Al mapear roles de manera estructurada, el equipo puede descubrir requerimientos clave de diferentes poblaciones de usuarios y asegurarse de satisfacer a los perfiles que realmente determinarán el éxito del proyecto.

Técnicas de Modelado - Personas y Personajes Extremos



Técnica de Personas - User Personas

- Consiste en crear una representación ficticia e imaginario de un rol de usuario específico.
- En lugar de un perfil abstracto, se le asigna un trasfondo de vida para humanizar al usuario final y ayudar al equipo a debatir funcionalidades en base a necesidades reales.



Personajes Extremos - Extreme Characters

- Técnica donde se diseña o evalúa el sistema

- considerando usuarios con personalidades, contexto o comportamientos altamente exagerados.
 - Permite descubrir requerimientos que de otra forma pasarían totalmente desapercibidos

Usuarios Representantes o Proxies

- Son **Intermediarios** que provienen de áreas del negocio o son expertos
 - Sirven para escribir, detallar y priorizar las User Stories en el día a día del proyecto ante la ausencia de usuarios de carne y hueso
 - El **Riesgo** es que cada Representante tiene sus propios sesgos intereses que pueden diferir de los del usuario real
-

Ciclo de Vida, Calidad y Criterio de Aceptación

•

Criterios de Aceptación y Pruebas de Aceptación

- Son la representación física de los detalles conversacionales y las expectativas específicas del cliente para una User Story
- Son las **Condiciones de Satisfacción** que el software debe cumplir antes de considerarse terminado
- Sirven para establecer límites claros en una User Story, unificar la visión del equipo, derivar pruebas y definir
- cuándo detener el desarrollo en una funcionalidad.

Proceso Colaborativo de Definición

1. Durante la conversación se escriben anotaciones o notas de pruebas en el reverso de la tarjeta de la Story, definiendo la intención (qué) y no la solución técnica (cómo).
2. Al momento de implementar las notas se convierten en pruebas de aceptación reales (idealmente

- automatizables) para verificar de forma constante que la Story fue codificada correctamente.

El Modelo INVEST - Calidad de las User Stories

- **I - Independent** indica que las historias deben poder implementarse en cualquier orden, evitando dependencias mutuas para facilitar la priorización y planificación del Backlog.
- **N - Negotiable** indica que no son contratos cerrados, sino que permite que los detalles finos surjan y se negocien en la diaria entre desarrolladores y clientes.
- **V - Valuable** indica que cada historia debe aportar un valor claro y visible para el cliente, debiendo evitarse las historias de carácter puramente tecnológico/técnico.
- **E - Estimatable** indica que el equipo debe ser capaz de prever el tamaño y esfuerzo de la historia, permitiendo priorizar las Stories y armar un ranking basado en la relación costo-beneficio
- **S - Small** indica que las Stories deben tener el tamaño adecuado para ser desarrolladas, probadas y consumidas por completo dentro de una sola iteración.
- **T - Testable** indica que debe ser posible demostrar y verificar mediante pruebas que la historia funciona correctamente para poder saber con certeza cuando se termina de desarrollar.

Definition of Readiness (DoR) vs Definition of Done (DoD)

○

Definition of Readiness (DoR)

- Es el acuerdo explícito del equipo que establece los requisitos de calidad y claridad que debe cumplir la User Story antes de ser aceptada en la reunión de planificación.
 - Evita que el equipo comience a desarrollar Stories ambiguas o mal definidas, reduciendo la incertidumbre y los bloqueos durante la iteración
-

Definition of Done (DoD)

- Es el acuerdo común entre el equipo y el Product

Owner que detalla qué significa que una Story esté 100% terminada

- Exige que el código esté escrito, probado por encargados de pruebas, documentado, integrado y listo para ser entregado.
 - Asegura un estándar de calidad interna no negociable para cada incremento potencialmente entregable, eliminado el autoengaño del desarrollador y la acumulación de deuda técnica al final de cada ciclo.
-

Gestión de Incertidumbre y Mitigación de Riesgos (Spikes)

.

Spikes (Técnicos y Funcionales) - Definición y Propósito

- Es un tipo especial de User Story que se utiliza para eliminar el riesgo y la incertidumbre de otra User Story.
- Consiste en una investigación, experimento rápido o prototipo que realiza el equipo para obtener el conocimiento necesario antes de comprometerse a desarrollar la
- funcionalidad real

Spikes Técnicos

- Son investigaciones enfocadas en explorar la viabilidad técnica y los enfoques dentro del dominio de la solución
- Se utilizan para evaluar la performance potencial de una arquitectura, tomar decisiones de hacer o comprar un componente o probar una nueva tecnología.
-

Spikes Funcionales

- Son investigaciones enfocadas en resolver la incertidumbre sobre cómo el usuario interactuará con el sistema para alcanzar el beneficio esperado.
- Sirven para definir flujos de pantallas, interacciones

- de usuario y reglas de negocio complejas, para los cuales se suelen usar generalmente prototipos.

Lineamientos Clave para Utilizar las Spikes como Excepción

1. Se debe **Aceptar la Incertidumbre del Día a Día**, poniendo el foco en aprender a resolver equivocándose y no abriendo una Spike por cada duda menor.
2. Se deben **Reservar para Incógnitas Críticas y Grandes** que involucren dudas tecnológicas o de negocio de gran escala que hagan imposible que el equipo pueda planificar el trabajo
3. Se deben **Usar como Última Opción** luego de dividir la Story en partes más pequeñas y manejables.
4. Se debe usar un **Timeboxing** (límite de tiempo) para evitar que el equipo caiga en la parálisis por análisis y la Spike sea estimable
5. Se deben **Separar en Diferentes Iteraciones** la resolución de la Spike y el desarrollo de la User Story, a menos que la Spike sea muy pequeña.
6. Debe ser **Estimable, Demostrable y Aceptable**, pudiendo demostrarse físicamente al final del Sprint y contar con criterios para que el cliente lo acepte como completado